

Graphs: Topological Sort and Directed Graph Problems

A Problem-Solving Companion to Striver's A2Z and SDE Sheets

Maruri Saikiran

June 30, 2026

Contents

1	Graphs: Topological Sort and Directed Graph Problems	1
1.1	Ordering a DAG	1
1.2	Topological Sort using DFS	3
1.3	Topological Sort using Kahn's Algorithm (BFS)	4
1.4	Detect Cycle in a Directed Graph	6
1.5	Course Schedule I	8
1.6	Course Schedule II	10
1.7	Eventual Safe States	11
1.8	Alien Dictionary	13

Chapter 1

Graphs: Topological Sort and Directed Graph Problems

A directed graph adds a constraint plain traversal never had to think about: order. Topological sort answers exactly one question – in what sequence can these tasks be done so that every prerequisite comes before what depends on it – and that single question, once answered, turns out to be the engine behind cycle detection, safe-state analysis, and even reconstructing an alien alphabet from sorted words.

1.1 Ordering a DAG

A **topological sort** of a Directed Acyclic Graph (DAG) is a linear ordering of its nodes such that for every directed edge $u \rightarrow v$, u appears **before** v in the ordering. Such an ordering exists if and only if the graph has **no cycle** – a cycle would force some node to both precede and follow itself, which is impossible in a linear order.

Recognizing a Topological Sort Problem

“Task scheduling with prerequisites,” “compilation order,” “course ordering,” or any phrase implying a valid sequence must respect a set of directed before/after constraints all point directly at topological sort.

Two Ways to Compute It

- **DFS-based (Section 1.2):** run DFS; the moment a node has no more unvisited neighbors to explore (it is fully “finished”), push it onto a stack. The stack, popped from top to bottom, is a valid topological order – a node finishes *after* everything reachable from it, so it naturally ends up before its dependents once the stack is reversed.
- **Kahn’s Algorithm / BFS-based (Section 1.3):** repeatedly remove nodes with **in-degree zero** (no remaining unprocessed prerequisites), decrementing the in-degree of their neighbors as they’re removed – directly simulating “do everything whose prerequisites are already done, then repeat.”

What This Chapter Covers

- Topological Sort, both algorithms (Sections 1.2–1.3).
- Detect Cycle in a Directed Graph (Section 1.4): a directed-graph adaptation of the parent-tracking idea from the BFS/DFS chapter, plus a free cycle check baked into Kahn’s algorithm itself.
- Course Schedule I and II (Sections 1.5–1.6): direct applications of cycle

detection and topological sort to a course-prerequisite framing.

- **Eventual Safe States (Section 1.7):** a three-color DFS variant identifying nodes that can never reach a cycle.
- **Alien Dictionary (Section 1.8):** building a graph from word-comparison constraints, then running topological sort to recover a letter ordering.

1.2 Topological Sort using DFS

Problem Statement. Given a Directed Acyclic Graph (DAG) with n nodes, return a valid topological ordering – a linear arrangement where every edge $u \rightarrow v$ has u appearing before v .

Pattern Recognition

“Order nodes so every prerequisite comes first” is solved by DFS with one addition: push a node onto a **stack** the moment it **finishes** (every neighbor it can reach has already been fully explored). Popping the stack from top to bottom yields a valid topological order.

Why finish-time ordering works

When DFS finishes exploring node u entirely (including everything reachable from it) before pushing u onto the stack, every node v reachable from u has *already* been pushed (it must finish before u does, since u ’s own finish waits on its recursive calls returning). So v ends up **lower** in the stack than u – and popping top-to-bottom visits u before v , exactly the required order for edge $u \rightarrow v$.

Algorithm

1. Maintain a *visited* array and an empty stack.
2. DFS from every unvisited node: recurse into all unvisited neighbors first, and only **after** that recursion returns (the node is now fully finished), push the current node onto the stack.
3. Pop the entire stack to produce the topological order.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited, stack<int>&
   finishOrder) {
5      visited[node] = true;
6      for (int neighbor : adj[node]) {
7          if (!visited[neighbor]) {
8              dfs(neighbor, adj, visited, finishOrder);
9          }
10     }
11     finishOrder.push(node); // push only after all neighbors are done
12 }
13
14 vector<int> topoSortDFS(int n, vector<vector<int>>& adj) {
15     vector<bool> visited(n, false);
16     stack<int> finishOrder;
17
18     for (int i = 0; i < n; i++) {
19         if (!visited[i]) {
20             dfs(i, adj, visited, finishOrder);
21         }
22     }
23
24     vector<int> result;
```

```

25     while (!finishOrder.empty()) {
26         result.push_back(finishOrder.top());
27         finishOrder.pop();
28     }
29     return result;
30 }
31
32 int main() {
33     int n = 6;
34     vector<vector<int>> adj(n);
35     adj[5] = {2, 0}; adj[4] = {0, 1}; adj[2] = {3}; adj[3] = {1};
36
37     for (int x : topoSortDFS(n, adj)) cout << x << " ";
38     cout << endl; // 5 4 2 3 1 0 (one valid order)
39     return 0;
40 }

```

Dry run

Setup. Take edges $5 \rightarrow 2$, $5 \rightarrow 0$, $4 \rightarrow 0$, $4 \rightarrow 1$, $2 \rightarrow 3$, $3 \rightarrow 1$.

DFS from 0: no outgoing edges, finishes immediately, push 0. Stack: [0]. DFS from 1: no outgoing edges, push 1. Stack: [0, 1]. DFS from 2: recurse to 3; 3 recurses to 1 (already visited, skip); 3 finishes, push 3. Stack: [0, 1, 3]. Back at 2: finishes, push 2. Stack: [0, 1, 3, 2]. DFS from 4: neighbors 0, 1 both visited; finishes, push 4. Stack: [0, 1, 3, 2, 4]. DFS from 5: neighbors 2, 0 both visited; finishes, push 5. Stack: [0, 1, 3, 2, 4, 5]. Popping top to bottom: 5, 4, 2, 3, 1, 0 – every edge ($5 \rightarrow 2$, $5 \rightarrow 0$, $4 \rightarrow 0$, $4 \rightarrow 1$, $2 \rightarrow 3$, $3 \rightarrow 1$) indeed has its source appearing before its destination.

Complexity

Time Complexity. $O(V + E)$ – standard DFS traversal cost. **Space Complexity.** $O(V)$ for *visited*, the stack, and the recursion stack.

Takeaway

“Push on finish, read the stack top-down” is a clean, purely local rule – no node needs any global information about the rest of the graph to know when to push itself, only that its own DFS call has returned. This locality is what makes the DFS approach to topological sort so simple to implement correctly.

1.3 Topological Sort using Kahn’s Algorithm (BFS)

Problem Statement. Same as Section 1.2: return a valid topological ordering of a DAG – this time using an in-degree-based BFS instead of DFS finish times.

Pattern Recognition

“Repeatedly do whatever has no remaining prerequisites” is solved directly: track each node’s **in-degree** (number of unprocessed incoming edges); start with every in-degree-0 node in a queue; each time a node is processed, decrement its neighbors’ in-degrees, and any neighbor that drops to 0 joins the queue.

Why this directly produces a valid order

A node can only be safely placed in the output once **every** prerequisite pointing into it has already been placed – which is exactly what in-degree 0 means at the moment a node is dequeued (all of its incoming edges have already been “consumed” by their source nodes being processed earlier). This makes Kahn’s algorithm a direct simulation of the real-world process the topological sort is modeling: do everything that’s ready, which unlocks more things to become ready, repeat.

Algorithm

1. Compute the in-degree of every node by scanning all edges.
2. Push every node with in-degree 0 into a queue.
3. While the queue is non-empty: pop a node, append it to the result; for each of its neighbors, decrement their in-degree, and push any neighbor whose in-degree reaches 0.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> topoSortKahn(int n, vector<vector<int>>& adj) {
5      vector<int> inDegree(n, 0);
6      for (int u = 0; u < n; u++) {
7          for (int v : adj[u]) inDegree[v]++;
8      }
9
10     queue<int> q;
11     for (int i = 0; i < n; i++) {
12         if (inDegree[i] == 0) q.push(i);
13     }
14
15     vector<int> result;
16     while (!q.empty()) {
17         int node = q.front();
18         q.pop();
19         result.push_back(node);
20
21         for (int neighbor : adj[node]) {
22             inDegree[neighbor]--;
23             if (inDegree[neighbor] == 0) q.push(neighbor);
24         }
25     }
26     return result;
27 }
28
29 int main() {
30     int n = 6;
31     vector<vector<int>> adj(n);
32     adj[5] = {2, 0}; adj[4] = {0, 1}; adj[2] = {3}; adj[3] = {1};
33
34     for (int x : topoSortKahn(n, adj)) cout << x << " ";
35     cout << endl; // 4 5 2 0 3 1 (one valid order)
36     return 0;
37 }

```

Dry run

Setup. Same graph as Section 1.2: edges $5 \rightarrow 2, 5 \rightarrow 0, 4 \rightarrow 0, 4 \rightarrow 1, 2 \rightarrow 3, 3 \rightarrow 1$.

In-degrees: 0 has in-degree 2 (from 5, 4); 1 has in-degree 2 (from 4, 3); 2 has in-degree 1 (from 5); 3 has in-degree 1 (from 2); 4, 5 have in-degree 0. Initial queue: [4, 5]. Pop 4: result [4]; decrement $0 \rightarrow 1, 1 \rightarrow 1$ (neither reaches 0 yet). Pop 5: result [4, 5]; decrement $2 \rightarrow 0$ (push 2), $0 \rightarrow 0$ (push 0). Queue: [2, 0]. Pop 2: result [4, 5, 2]; decrement $3 \rightarrow 0$ (push 3). Pop 0: result [4, 5, 2, 0]; no outgoing edges. Pop 3: result [4, 5, 2, 0, 3]; decrement $1 \rightarrow 0$ (push 1). Pop 1: result [4, 5, 2, 0, 3, 1]. Final order: 4, 5, 2, 0, 3, 1 – a different but equally valid ordering from the DFS approach’s 5, 4, 2, 3, 1, 0 (topological order is rarely unique).

Complexity

Time Complexity. $O(V + E)$ – computing in-degrees scans every edge once; the BFS itself also visits every node and edge once. **Space Complexity.** $O(V)$ for the in-degree array and the queue.

Takeaway

Kahn’s algorithm has one advantage DFS’s approach doesn’t offer for free: if the queue empties before every node has been processed, the remaining unprocessed nodes are necessarily stuck in a cycle (nothing in a cycle can ever reach in-degree 0, since each node in the cycle is always waiting on another node in the same cycle). This makes Kahn’s algorithm double as a **cycle detector** in directed graphs, used directly in the next section.

1.4 Detect Cycle in a Directed Graph

Problem Statement. Given a directed graph, determine whether it contains a cycle.

Pattern Recognition

Unlike the undirected case (the Graph BFS and DFS Problems chapter’s Detect Cycle in an Undirected Graph section), a single “parent” check is not enough here: in a *directed* graph, reaching an already-visited node is perfectly normal (two different paths can legitimately converge on the same node without any cycle). What signals a true cycle is reaching a node that is still **on the current DFS path** – tracked with a second array, separate from *visited*, marking only nodes currently “in progress.”

Why *visited* alone is insufficient here

Consider $A \rightarrow B, A \rightarrow C, B \rightarrow D, C \rightarrow D$: node D is visited twice (once via B , once via C), but there is no cycle – both are legitimate converging paths. The distinguishing feature of an actual cycle is a **back edge**: an edge pointing to a node that is an **ancestor** of the current node in the DFS recursion tree (i.e., still actively on the call stack), not merely visited at some earlier, already-concluded point in the traversal.

Approach 1: DFS with a Path-Tracking Array

```
1 #include <bits/stdc++.h>
2 using namespace std;
```

```

3
4 bool dfsHasCycle(int node, vector<vector<int>>& adj, vector<bool>& visited,
5                 vector<bool>& pathVisited) {
6     visited[node] = true;
7     pathVisited[node] = true;
8
9     for (int neighbor : adj[node]) {
10        if (!visited[neighbor]) {
11            if (dfsHasCycle(neighbor, adj, visited, pathVisited)) return true;
12        } else if (pathVisited[neighbor]) {
13            return true; // back edge to a node still on the current path
14        }
15    }
16    pathVisited[node] = false; // backtrack: leaving this node's path
17    return false;
18 }
19
20 bool hasCycleDFS(int n, vector<vector<int>>& adj) {
21     vector<bool> visited(n, false), pathVisited(n, false);
22     for (int i = 0; i < n; i++) {
23         if (!visited[i] && dfsHasCycle(i, adj, visited, pathVisited)) {
24             return true;
25         }
26     }
27     return false;
28 }
29
30 int main() {
31     int n = 3;
32     vector<vector<int>> adj(n);
33     adj[0] = {1}; adj[1] = {2}; adj[2] = {0};
34
35     cout << (hasCycleDFS(n, adj) ? "true" : "false") << endl; // true
36     return 0;
37 }

```

Approach 1 Complexity

Time. $O(V + E)$. **Space.** $O(V)$ for *visited*, *pathVisited*, and the recursion stack.

Approach 2: Kahn's Algorithm (BFS)

As noted at the close of Section 1.3, if Kahn's algorithm processes fewer than n nodes before its queue empties, the unprocessed nodes are necessarily trapped in a cycle (a cycle's nodes can never reach in-degree 0, since each waits on another node within the same cycle).

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool hasCycleKahn(int n, vector<vector<int>>& adj) {
5     vector<int> inDegree(n, 0);
6     for (int u = 0; u < n; u++) {
7         for (int v : adj[u]) inDegree[v]++;
8     }
9
10    queue<int> q;
11    for (int i = 0; i < n; i++) {
12        if (inDegree[i] == 0) q.push(i);
13    }
14
15    int processed = 0;

```

```

16 while (!q.empty()) {
17     int node = q.front();
18     q.pop();
19     processed++;
20
21     for (int neighbor : adj[node]) {
22         inDegree[neighbor]--;
23         if (inDegree[neighbor] == 0) q.push(neighbor);
24     }
25 }
26 return processed != n; // fewer than n processed => cycle exists
27 }
28
29 int main() {
30     int n = 3;
31     vector<vector<int>> adj(n);
32     adj[0] = {1}; adj[1] = {2}; adj[2] = {0};
33
34     cout << (hasCycleKahn(n, adj) ? "true" : "false") << endl; // true
35     return 0;
36 }

```

Dry run

Setup. Take the 3-cycle $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$.

DFS: $dfs(0)$: $visited[0] = \text{true}$, $pathVisited[0] = \text{true}$. Neighbor 1: unvisited, recurse. $dfs(1)$: visited, $pathVisited$ both true for 1. Neighbor 2: unvisited, recurse. $dfs(2)$: visited, $pathVisited$ both true for 2. Neighbor 0: **already visited, and $pathVisited[0] = \text{true}$** (still on the current path) – cycle detected, return **true**.

Kahn's: in-degrees all equal 1 (each node has exactly one incoming edge from the cycle). No node starts with in-degree 0, so the queue is empty from the start; $processed = 0 \neq 3 = n$ – cycle detected immediately, without even examining a single edge.

Approach 2 Complexity

Time. $O(V + E)$. **Space.** $O(V)$ for in-degrees and the queue.

Takeaway

Directed-graph cycle detection needs to distinguish “visited at some point” from “visited and still active on this path” – the *pathVisited* array (cleared on backtrack) is what undirected cycle detection’s simpler parent check generalizes into once edges have direction. Kahn’s algorithm sidesteps this distinction entirely by checking a global count instead, which is part of why it is often preferred for this specific check.

1.5 Course Schedule I

Problem Statement. Given *numCourses* and a list of prerequisite pairs $[a, b]$ (meaning course a requires course b to be completed first), determine whether it is possible to finish all courses.

Pattern Recognition

“Can all tasks be completed given prerequisite constraints” is a direct restatement of Section 1.4: it is possible to finish all courses if and only if the prerequisite graph (edge

$b \rightarrow a$ for every pair $[a, b]$, since b must come first) contains **no cycle** – a cycle of prerequisites would mean some course indirectly requires itself.

Reduction

Build a directed graph with $numCourses$ nodes, adding an edge $b \rightarrow a$ for every prerequisite pair $[a, b]$ (course b must be completed before a , so the edge points from prerequisite to dependent). The answer is simply $\neg hasCycle(numCourses, adj)$, using either cycle-detection approach from Section 1.4 unchanged.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool dfsHasCycle(int node, vector<vector<int>>& adj, vector<bool>& visited,
5                 vector<bool>& pathVisited) {
6     visited[node] = true;
7     pathVisited[node] = true;
8
9     for (int neighbor : adj[node]) {
10        if (!visited[neighbor]) {
11            if (dfsHasCycle(neighbor, adj, visited, pathVisited)) return true;
12        } else if (pathVisited[neighbor]) {
13            return true;
14        }
15    }
16    pathVisited[node] = false;
17    return false;
18 }
19
20 bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
21     vector<vector<int>> adj(numCourses);
22     for (auto& p : prerequisites) {
23         int a = p[0], b = p[1];
24         adj[b].push_back(a); // b must come before a
25     }
26
27     vector<bool> visited(numCourses, false), pathVisited(numCourses, false);
28     for (int i = 0; i < numCourses; i++) {
29         if (!visited[i] && dfsHasCycle(i, adj, visited, pathVisited)) {
30             return false;
31         }
32     }
33     return true;
34 }
35
36 int main() {
37     vector<vector<int>> prerequisites = {{1, 0}, {0, 1}};
38     cout << (canFinish(2, prerequisites) ? "true" : "false") << endl; // false
39     return 0;
40 }

```

Dry run

Setup. Take $numCourses = 2$, $prerequisites = [[1, 0], [0, 1]]$.

Edge from pair $[1, 0]$: $0 \rightarrow 1$ (course 0 before course 1). Edge from pair $[0, 1]$: $1 \rightarrow 0$ (course 1 before course 0). This is a 2-cycle: $0 \rightarrow 1 \rightarrow 0$. Cycle detection immediately finds the back edge, so *canFinish* returns **false** – correctly, since each course requires the other to be completed first, an impossible circular dependency.

Complexity

Time Complexity. $O(V + E)$, inherited unchanged from cycle detection. **Space Complexity.** $O(V + E)$ for the adjacency list, plus $O(V)$ for the cycle-detection arrays.

Takeaway

Recognizing that “can all tasks be completed” is a cycle-detection question in disguise is the entire difficulty of this problem – once that reduction is seen, no new algorithm is needed at all.

1.6 Course Schedule II

Problem Statement. Given *numCourses* and prerequisite pairs $[a, b]$ as in Section 1.5, return **one valid order** to take all courses, or an empty array if no valid order exists.

Pattern Recognition

“Return a valid order respecting prerequisites” is a direct restatement of topological sort (Sections 1.2–1.3): build the same prerequisite graph as Course Schedule I, then run either topological sort algorithm. Kahn’s algorithm is the natural choice here since it detects the no-valid-order case (a cycle) **for free**, via the same processed-count check from Section 1.4.

Reduction

Build the graph identically to Course Schedule I ($b \rightarrow a$ for every pair $[a, b]$), run Kahn’s algorithm, and return the resulting order – but only if every node was processed; otherwise return an empty array (a cycle means no valid order exists).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
5     vector<vector<int>> adj(numCourses);
6     vector<int> inDegree(numCourses, 0);
7
8     for (auto& p : prerequisites) {
9         int a = p[0], b = p[1];
10        adj[b].push_back(a);
11        inDegree[a]++;
12    }
13
14    queue<int> q;
15    for (int i = 0; i < numCourses; i++) {
16        if (inDegree[i] == 0) q.push(i);
17    }

```

```

18     vector<int> order;
19     while (!q.empty()) {
20         int node = q.front();
21         q.pop();
22         order.push_back(node);
23
24         for (int neighbor : adj[node]) {
25             inDegree[neighbor]--;
26             if (inDegree[neighbor] == 0) q.push(neighbor);
27         }
28     }
29 }
30
31 if ((int)order.size() != numCourses) return {}; // cycle: no valid order
32 return order;
33 }
34
35 int main() {
36     vector<vector<int>> prerequisites = {{1, 0}, {2, 0}, {3, 1}, {3, 2}};
37     vector<int> order = findOrder(4, prerequisites);
38     for (int x : order) cout << x << " ";
39     cout << endl; // 0 1 2 3 (one valid order)
40     return 0;
41 }

```

Dry run

Setup. Take $numCourses = 4$, $prerequisites = [[1, 0], [2, 0], [3, 1], [3, 2]]$ (course 3 needs both 1 and 2; both 1 and 2 need 0).

Edges: $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 3$. In-degrees: 0:0, 1:1, 2:1, 3:2. Initial queue: [0]. Pop 0: order [0]; decrement $1 \rightarrow 0$ (push), $2 \rightarrow 0$ (push). Queue: [1, 2]. Pop 1: order [0, 1]; decrement $3 \rightarrow 1$ (not 0 yet). Pop 2: order [0, 1, 2]; decrement $3 \rightarrow 0$ (push). Pop 3: order [0, 1, 2, 3]. Size $4 = numCourses$: valid order found.

Complexity

Time Complexity. $O(V + E)$, inherited unchanged from Kahn's algorithm. **Space Complexity.** $O(V + E)$ for the adjacency list and in-degree array.

Takeaway

Course Schedule I and II together demonstrate the cleanest possible illustration of this chapter's two core tools applied side by side: the exact same prerequisite graph answers "is it possible" via cycle detection and "what is an order" via topological sort, with no additional derivation needed for either once the graph is built.

1.7 Eventual Safe States

Problem Statement. Given a directed graph, a node is **safe** if every possible path starting from it eventually leads to a **terminal node** (a node with no outgoing edges) – meaning no path from it can ever enter a cycle. Return all safe nodes, sorted.

Pattern Recognition

“Never gets stuck in a cycle, on any path” extends Section 1.4’s *pathVisited* idea into a full **three-state** DFS: a node is currently **being explored** (on the active path – analogous to *pathVisited*), **fully confirmed safe**, or **not yet visited**. Reaching a node still being explored signals a cycle, exactly as before – the new piece is caching the confirmed **safe/unsafe** verdict for every node once decided, so each node’s full reachability is resolved only once.

Why a single boolean *visited* array is not enough

A node might be reached by DFS, fully resolved as safe or unsafe, and then reached *again* via a different path later in the traversal – a plain *visited* array would either skip re-deriving its answer (fine, if it remembers the answer) or, worse, treat it as “currently on the path” if visited and not-yet-finished are conflated into one state. Three explicit states – 0 = *unvisited*, 1 = *visiting*, 2 = *safe* – avoid this ambiguity entirely.

Algorithm

1. For each node, maintain a state: 0 (unvisited), 1 (currently being explored – on the active DFS path), or 2 (confirmed safe).
2. *dfs(node)*: if state is 2, return **true** (already confirmed safe). If state is 1, return **false** (currently on the path – this is a back edge, a cycle). Otherwise, mark state 1 (entering); recurse into every neighbor – if *any* neighbor’s DFS returns **false**, this node is unsafe, return **false** immediately. If every neighbor returns **true**, mark state 2 (confirmed safe) and return **true**.
3. Collect every node whose DFS returns **true**.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool dfs(int node, vector<vector<int>>& adj, vector<int>& state) {
5      if (state[node] == 2) return true;
6      if (state[node] == 1) return false;
7
8      state[node] = 1; // currently being explored
9      for (int neighbor : adj[node]) {
10         if (!dfs(neighbor, adj, state)) return false;
11     }
12     state[node] = 2; // confirmed safe
13     return true;
14 }
15
16 vector<int> eventualSafeNodes(int n, vector<vector<int>>& adj) {
17     vector<int> state(n, 0);
18     vector<int> result;
19
20     for (int i = 0; i < n; i++) {
21         if (dfs(i, adj, state)) {
22             result.push_back(i);
23         }
24     }

```

```

25     return result; // already sorted, since nodes are processed in order
26 }
27
28 int main() {
29     int n = 7;
30     vector<vector<int>> adj(n);
31     adj[0] = {1, 2}; adj[1] = {2, 3}; adj[2] = {5}; adj[3] = {0};
32     adj[4] = {5}; adj[5] = {}; adj[6] = {};
33
34     for (int x : eventualSafeNodes(n, adj)) cout << x << " ";
35     cout << endl; // 2 4 5 6
36     return 0;
37 }

```

Dry run

Setup. Take the graph above: $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 5, 3 \rightarrow 0, 4 \rightarrow 5$ (5, 6 terminal).

$dfs(0)$: state[0] = 1. Neighbor 1: $dfs(1)$: state[1] = 1. Neighbor 2: $dfs(2)$: state[2] = 1. Neighbor 5: $dfs(5)$: no neighbors, state[5] = 2, return **true**. Back at 2: state[2] = 2, return **true**. Back at 1: neighbor 3: $dfs(3)$: state[3] = 1. Neighbor 0: state[0] = 1 – **still being explored** – return **false**. So $dfs(3)$ returns **false**, making $dfs(1)$ return **false** (it has an unsafe neighbor), which in turn makes $dfs(0)$ return **false**. Continuing with $i = 2$: already state 2, returns **true**, added. $i = 4$: $dfs(4)$: neighbor 5, already safe (state 2); 4 becomes safe. $i = 5, 6$: already safe / trivially safe (no neighbors). Final safe nodes: 2, 4, 5, 6 – matching the expected output, while 0, 1, 3 are correctly excluded since they all lie on or lead into the cycle $0 \rightarrow 1 \rightarrow 3 \rightarrow 0$.

Complexity

Time Complexity. $O(V + E)$ – each node’s DFS result is computed and cached exactly once (the state-2 check short-circuits any repeat work). **Space Complexity.** $O(V)$ for the state array and the recursion stack.

Takeaway

The white/gray/black three-coloring scheme used here is the general form of cycle-aware DFS: “gray” (in-progress) plays the role *pathVisited* played in plain cycle detection, while “black” (confirmed done) adds memoization on top, caching each node’s final verdict so a graph with many converging paths is still only ever analyzed once per node.

1.8 Alien Dictionary

Problem Statement. Given a list of words sorted lexicographically according to the rules of an unknown alien language with k distinct lowercase letters, determine a valid ordering of the alphabet. Return an empty string if no valid ordering exists.

Pattern Recognition

This closing problem of the chapter is a two-step composition: **build** a directed graph of letter-ordering constraints by comparing each pair of **adjacent** words in the list, then **run topological sort** (Sections 1.2–1.3) on that graph to recover the alphabet order. The graph-building step is the only new idea; the ordering step is unchanged from earlier in this chapter.

Extracting constraints from adjacent word pairs

For two adjacent words w_1, w_2 in the sorted list, scan character by character until the **first** position where they differ. If $w_1[i] \neq w_2[i]$, that single comparison reveals exactly one constraint: $w_1[i]$ comes before $w_2[i]$ in the alien alphabet (edge $w_1[i] \rightarrow w_2[i]$) – and nothing more can be concluded from this pair, since the relative order of any later characters is unconstrained by this comparison. If no differing position is found within the shorter word's length, the ordering is **invalid** exactly when w_1 is *longer* than w_2 (a longer word cannot correctly precede its own prefix in sorted order).

Algorithm

1. Initialize an empty adjacency structure over the (at most 26) distinct letters appearing in the words.
2. For every adjacent pair of words, find the first differing character and add the corresponding edge; if no differing character exists and the first word is longer, return "" immediately (invalid).
3. Run topological sort (Kahn's algorithm is convenient here, since it naturally detects a cycle – contradictory constraints – by an incomplete processed count) over the graph of letters.
4. If a cycle is detected, return ""; otherwise return the topological order as a string.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  string alienOrder(vector<string>& words) {
5      unordered_set<char> letters;
6      for (string& w : words) for (char c : w) letters.insert(c);
7
8      unordered_map<char, unordered_set<char>> adj;
9      unordered_map<char, int> inDegree;
10     for (char c : letters) inDegree[c] = 0;
11
12     for (int i = 0; i + 1 < (int)words.size(); i++) {
13         string& w1 = words[i];
14         string& w2 = words[i + 1];
15         int minLen = min(w1.size(), w2.size());
16         bool foundDiff = false;
17
18         for (int j = 0; j < minLen; j++) {
19             if (w1[j] != w2[j]) {
20                 if (adj[w1[j]].find(w2[j]) == adj[w1[j]].end()) {
21                     adj[w1[j]].insert(w2[j]);
22                     inDegree[w2[j]]++;
23                 }
24                 foundDiff = true;
25                 break;
26             }
27         }
28         if (!foundDiff && w1.size() > w2.size()) return "";
29     }
30
31     queue<char> q;
32     for (char c : letters) {

```

```

33     if (inDegree[c] == 0) q.push(c);
34 }
35
36 string order;
37 while (!q.empty()) {
38     char c = q.front();
39     q.pop();
40     order += c;
41
42     for (char neighbor : adj[c]) {
43         inDegree[neighbor]--;
44         if (inDegree[neighbor] == 0) q.push(neighbor);
45     }
46 }
47
48 return (int)order.size() == (int)letters.size() ? order : "";
49 }
50
51 int main() {
52     vector<string> words = {"wrt", "wrf", "er", "ett", "rftt"};
53     cout << alienOrder(words) << endl; // wertf
54     return 0;
55 }

```

Dry run

Setup. Take $words = [wrt, wrf, er, ett, rftt]$.

Pair (wrt,wrf): differ at index 2 (t vs f) – edge $t \rightarrow f$. Pair (wrf,er): differ at index 0 (w vs e) – edge $w \rightarrow e$. Pair (er,ett): differ at index 1 (r vs t) – edge $r \rightarrow t$. Pair (ett,rftt): differ at index 0 (e vs r) – edge $e \rightarrow r$. Graph: $t \rightarrow f, w \rightarrow e, r \rightarrow t, e \rightarrow r$. In-degrees: $w:0, e:1, r:1, t:1, f:1$. Kahn's: start $[w]$. Pop w : order w ; $e \rightarrow 0$, push. Pop e : order we ; $r \rightarrow 0$, push. Pop r : order wer ; $t \rightarrow 0$, push. Pop t : order $wert$; $f \rightarrow 0$, push. Pop f : order $wertf$. All 5 letters processed: valid order "wertf".

Complexity

Time Complexity. $O(\sum |w_i| + V + E)$ – comparing each adjacent word pair costs time proportional to the shorter word's length, and the topological sort afterward costs $O(V + E)$ over the (at most 26) letters and their constraints. **Space Complexity.** $O(V + E)$ for the letter graph, bounded by $O(26^2)$ in the worst case.

Takeaway

This chapter closes by demonstrating topological sort's full generality: the “nodes” and “edges” don't need to be given explicitly at all – here they are *derived* from a completely different kind of input (a sorted word list) via a simple pairwise-comparison rule, and once that translation step is done, every algorithm from earlier in this chapter (cycle detection bundled into Kahn's algorithm, the topological order itself) applies completely unchanged.